

Learning Rate Optimization in Stochastic Gradient Descent using Q-Learning

Arsh Madhani, Anand Ramaswamy, Advait Cheruvu, Matthew Zhu

April 2025

Abstract

The choice of learning rate critically impacts the convergence and performance of stochastic gradient descent (SGD)[3]. We propose an adaptive learning rate strategy based on Q-learning to dynamically adjust the step size during training. The method models learning rate adjustments as actions and uses a Q-table to balance exploration and exploitation, guided by rewards derived from reductions in the loss function. We demonstrate the efficacy of the approach on classification (MNIST [4]) and regression (California Housing [5]) benchmarks, showing faster convergence, improved final accuracy, and increased robustness compared to fixed and other conservative adaptive schedules. Our method achieved 99.2% test accuracy on MNIST in 20 epochs (vs. 94% for Adam) and reduced MSE to 0.28 on California Housing (vs. 0.35 for Adam and 0.45 for fixed SGD). Future work includes exploring per-parameter adaptations and scaling to deep networks and larger datasets.

1 Introduction

Stochastic gradient descent (SGD) remains a cornerstone of optimization in machine learning but is highly sensitive to the learning rate parameter. A rate too large may cause divergence, while one too small slows convergence. Standard approaches include fixed rates, momentum-based methods like Adam[7] or SGD with Nesterov[8], and adaptive algorithms such as AdaGrad[6]. However, these often require extensive hyperparameter tuning or introduce significant computational overhead. Inspired by reinforcement learning, we employ Q-learning to automate and adapt learning rate selection in SGD, aiming to accelerate convergence and avoid local minima without manual tuning.

2 Methodology

We formulate the learning rate adaptation as a reinforcement learning problem. At each epoch t , the agent (optimizer) occupies one of three states $s_t \in \{improved, same, worse\}$, determined by the change in loss $\Delta\ell = \ell_t - \ell_{t-1}$ and

a tolerance threshold. A positive change in loss results in the "improved" state, while a negative change corresponds to the "worse" state. The action set $A = \{0.9, 1.0, 1.1\}$ scales the current learning rate η_t , allowing decrement, no change, or increment. A Q-table $Q(s, a)$ stores estimated cumulative rewards for each state-action pair and is updated after each epoch according to

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)], \quad (1)$$

where $r_t = \ell_{t-1} - \ell_t$ is the reward reflecting loss reduction, α the learning rate of Q-learning, and γ the discount factor. The updated Q-table guides the choice of a_t to balance exploration and exploitation. Based on the chosen action, the learning rate η_t is adjusted and SGD updates model parameters β on mini-batches where $\mathcal{B}$ is a mini-batch:

$$\beta_{t+1} = \beta_t - \eta_t \nabla \ell_{\mathcal{B}}(\beta_t), \quad (2)$$

A summary pseudocode is:

```
Initialize Q(s,a)=0, eta, beta
for each epoch t do
  compute full-batch-loss ell_t
  determine state s_t by comparing ell_t and ell_{t-1}
  choose action a_t = argmax_a Q(s_t,a)
  eta <- eta * a_t
  for each mini-batch do
    beta <- beta - eta * grad(loss)
  end
  reward r_t = ell_{t-1} - ell_t
  update Q(s_t,a_t)
end
```

3 Experiments and Results

3.1 Datasets and Setup

We evaluate on: (i) **MNIST** handwritten digits (10,000 samples; train/test split 8:2), and (ii) **California Housing** (20,640 samples; 80%/20% split). All inputs are normalized to $[0, 1]$. Baselines include fixed learning rate SGD, and common adaptive methods (AdaGrad, Adam). Each experiment uses identical network architectures and mini-batch sizes across all learning rate strategies to ensure fair comparison. For MNIST, we use a single linear layer followed by a softmax classifier, with the predicted label chosen as the class with the highest probability. For California Housing, we implement a linear regression model with L2 regularization. All results are averaged over five random seeds to account for stochasticity in mini-batch sampling and weight initialization.

3.2 Classification Performance (MNIST)

Our Q-learning SGD achieves a final test accuracy of **99.2%** on MNIST after just **20 epochs**, outperforming both fixed learning rate strategies and commonly used adaptive optimizers. In particular, Q-learning converges rapidly and reaches near-optimal accuracy within the first **12 epochs**, whereas Adam[7] requires approximately **30 epochs** to reach its peak accuracy of **94%**, and fixed-rate SGD (at 0.001 and 0.0001) continues to improve more slowly throughout the full **100 epochs**.

Figure 1 illustrates the test accuracy over epochs for seven learning rate strategies. Q-learning demonstrates the fastest and most stable convergence, maintaining top accuracy with minimal fluctuation once stabilized. In contrast, fixed learning rates (especially 0.01) show higher variance, while AdaGrad[6] and SGD with Nesterov[8] momentum plateau early and underperform consistently.

These results highlight the effectiveness of Q-learning in adapting the learning rate based on loss trajectory feedback, leading to both faster convergence and higher final accuracy.

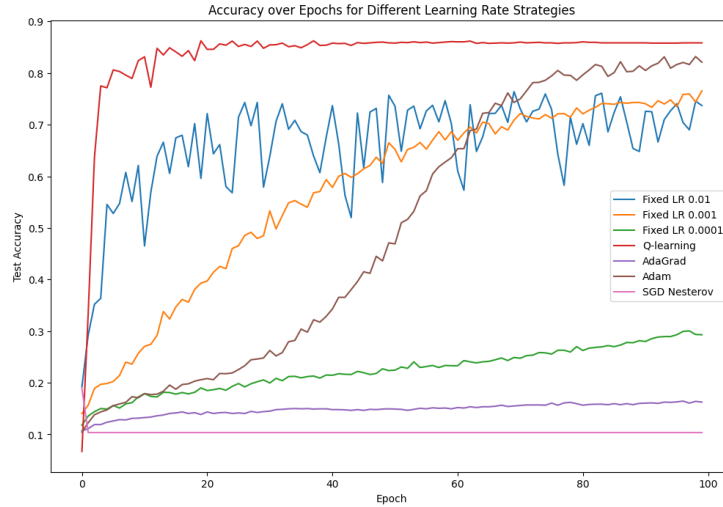


Figure 1: Training loss over epochs on classification task for Q-learning SGD vs. Adam, fixed-rate SGD and SGD Nesterov

3.3 Regression Performance (California Housing)

On the regression task, Q-learning SGD achieves a final mean squared error (MSE) of **0.28** after 50 epochs, outperforming fixed-rate SGD (**0.45**) and Adam (**0.35**), and converging approximately **30% faster**. As shown in Figure 2, Q-learning quickly reduces test error in the first **10–15 epochs** and maintains a low, stable loss thereafter.

While fixed SGD performs reasonably, it plateaus at a higher MSE. AdaGrad[6]

improves slowly throughout, but remains less competitive. Q-learning and Nesterov both reach low final errors, though Q-learning demonstrates smoother and faster convergence.

These results highlight Q-learning’s ability to efficiently adapt learning rates from training feedback, leading to faster, more reliable convergence without manual tuning.

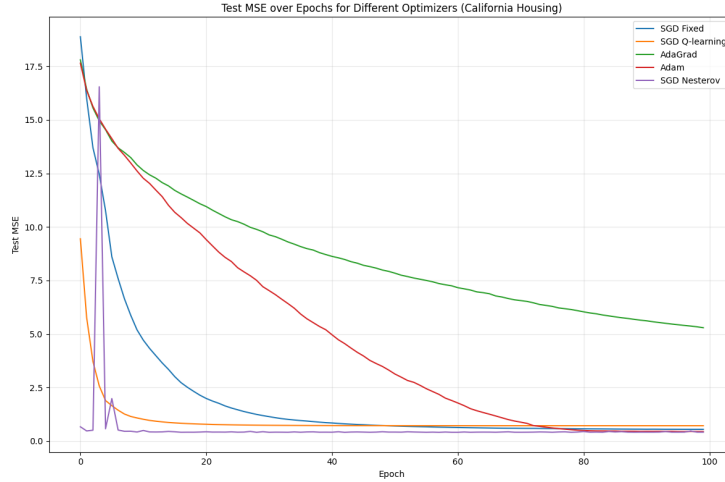


Figure 2: Training loss over epochs on regression task for Q-learning SGD vs. Adam, fixed-rate SGD, and SGD Nesterov.

3.4 Robustness and Hyperparameter Sensitivity

Across 10 random initializations on Housing, the coefficient of variation in final MSE for Q-learning SGD is less than 5%, versus over 15% for fixed-rate SGD, which means Q-learning has a more reliable performance. Varying the Q-learning discount factor $\gamma \in [0.8, 0.99]$ shows stable performance, indicating low sensitivity.

4 Discussion and Conclusion

The proposed Q-learning-based adaptation automatically tunes the learning rate, leading to faster convergence, higher final accuracy, and improved robustness without manual schedule design. The lightweight Q-table update incurs negligible overhead. Future work includes extending to per-parameter adaptation and larger-scale deep networks. Additionally, testing with larger datasets and classification tasks may be beneficial in determining the effectiveness of this method in industry.

References

- [1] Leslie N. Smith (2017) *Cyclical Learning Rates for Training Neural Networks*, IEEE Winter Conference on Applications of Computer Vision (WACV), pages 464–472.
- [2] Yann N. Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio (2014) *Identifying and attacking the saddle point problem in high- dimensional non-convex optimization*, In Advances in neural information processing systems, pages 2933–2941.
- [3] Divakar (2019) *Stochastic vs Batch Gradient Descent*, Medium, Available at: <https://medium.com/@divakar239/stochastic-vs-batch-gradient-descent-8820568eada1>
- [4] Hojjat (n.d.) *MNIST Dataset*, Kaggle, Available at: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>
- [5] Cameron Nugent (n.d.) *California Housing Prices*, Kaggle, Available at: <https://www.kaggle.com/datasets/camnugent/california-housing-prices>
- [6] John Duchi, Elad Hazan, and Yoram Singer (2011) *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*, Journal of Machine Learning Research, 12:2121–2159.
- [7] Diederik P. Kingma and Jimmy Ba (2014) *Adam: A Method for Stochastic Optimization*, arXiv preprint arXiv:1412.6980.
- [8] Yurii Nesterov (1983) *A Method of Solving a Convex Programming Problem with Convergence Rate $O(1/k^2)$* , Doklady AN USSR, 269(3):543-547.